K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Data Structures(ETCCDS202)

Assignment - 3

Assignment Title: Sorting Performance Analyzer (SPA)

Submitted By: NEHA

Submitted to : Dr. Neetu Chauhan

Roll Number: 2501730310

Program: B.Tech CSE (AI & ML)

Course Title : Data Structures

1. Introduction

This project focuses on the implementation and analysis of fundamental sorting algorithms using Python. The aim is to study how different sorting techniques behave under various input conditions and compare their practical performance with theoretical time complexities.

Three sorting algorithms were implemented from scratch:

- Insertion Sort
- Merge Sort
- Quick Sort

Their performance was tested on random, sorted, and reverse-sorted datasets.

2. Objectives

- Implement sorting algorithms without using built-in functions
- Analyze execution time for different inputs
- Compare algorithm efficiency
- Study stability and memory usage

3. Algorithms Implemented

3.1 Insertion Sort

- Time Complexity: $O(n^2)$
- Best Case: $O(n)$
- Space Complexity: $O(1)$
- Stable: Yes
- In-place: Yes

3.2 Merge Sort

- Time Complexity: $O(n \log n)$

- Space Complexity: O(n)
- Stable: Yes
- In-place: No

3.3 Quick Sort

- Average Time Complexity: O(n log n)
- Worst Case: $O(n^2)$
- Space Complexity: O(log n)
- Stable: No
- In-place: Yes

4. Dataset Description

The datasets used in this experiment include:

- Random arrays (sizes: 1000, 5000, 10000)
- Sorted arrays (sizes: 1000, 5000, 10000)
- Reverse-sorted arrays (sizes: 1000, 5000, 10000)

All datasets consist of integer values generated programmatically.

5. Performance Measurement

Execution time was measured using a common timing function to ensure fair comparison. Each algorithm was executed on a copy of the same dataset.

6. Results Table

Input Type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random	1000	50.579	6.665	3.677
Random	5000	1491.624	30.782	12.51
Random	10000	3517.779	51.639	25.455
Sorted	1000	0.381	4.403	170.565
Sorted	5000	1.301	18.842	1865.786
Sorted	10000	2.601	28.1	7402.617
Reverse	1000	135.339	5.745	106.467
Reverse	5000	1651.481	12.062	1241.105
Reverse	10000	6030.699	23.414	4509.773

---

7. Analysis

Random Data

Merge Sort and Quick Sort performed significantly better than Insertion Sort due to their O(n log n) complexity. Insertion Sort showed very high execution time for large datasets.

Sorted Data

Insertion Sort performed the fastest due to its best-case O(n) complexity. Quick Sort performed poorly because the pivot (last element) caused unbalanced partitions, leading to worst-case behavior.

Reverse-Sorted Data

Insertion Sort showed worst-case performance. Quick Sort also slowed down due to poor pivot selection. Merge Sort remained consistent across all inputs.

---

8. Theoretical vs Practical Comparison

The experimental results closely match theoretical time complexities:

- Insertion Sort is inefficient for large datasets
- Merge Sort provides consistent performance
- Quick Sort is efficient on average but sensitive to pivot choice

9. Stability and Memory Usage

Algorithm	Stable	In-place	Extra Space
Insertion Sort	Yes	Yes	No
Merge Sort	Yes	No	Yes
Quick Sort	No	Yes	Minimal

10. Conclusion

This project highlights the importance of selecting the appropriate sorting algorithm based on input data. Insertion Sort is efficient for small or nearly sorted datasets, while Merge Sort and Quick Sort are better suited for large datasets. Merge Sort provides consistent performance, whereas Quick Sort is faster on average but depends heavily on pivot selection.

The experimental results validate the theoretical time complexities and demonstrate real-world behavior of sorting algorithms.